LAB03 Debug và phân tích hiệu năng

Mai The Tam
MSSV: [chưa cung cấp] Nhóm/lớp: [chưa cung cấp]
Ngày thực hiện: 28/10/2026

Bài thực hành kiểm tra ứng dụng ARM bằng GDB remote, phân tích memory leak trên HOST bằng Valgrind, phân tích syscall và CPU bằng strace/perf, đồng thời đo wake-up latency của kernel HOST. Các kết luận dưới đây dựa trên log thực nghiệm đi kèm repo LOS201.

1 Môi trường và cách tái hiện

Thành phần	Giá trị
HOST	Ubuntu x86-64; Intel Core i5-8250U, 8 CPU logic
Kernel HOST	7.0.0-34-generic; PREEMPT_DYNAMIC, CONFIG_PREEMPT_LAZY=y; không phải PREEMPT_RT
TARGET	QEMU vexpress-a9, Cortex-A9, 2 vCPU, RAM 512 MiB, Linux 5.15.0
Compiler	GCC và arm-linux-gnueabihf-gcc 13.3.0
Công cụ	GDB/gdbserver 15.1, Valgrind 3.22.0, strace 6.8, perf 7.0.14, cyclictest 2.50, gnuplot 6.0
Biên dịch	GDB/memory: -g -O0. Profiling: -g -O1 -fno-omit-frame-pointer. Ứng dụng ARM link static.

Gdbserver và strace là gói Ubuntu armhf; loader/libc/libstdc++/libgcc ARM được bổ sung từ sysroot của toolchain. Guest nhận IP 10.0.2.15 qua DHCP. HOST 127.0.0.1:1234 được forward tới guest:1234. Kernel và rootfs nền lấy từ LAB01 và LAB02.

```
make -C embedded_lab3/apps buggy_app
bash embedded_lab3/scripts/pack.sh
bash embedded_lab3/scripts/run-qemu.sh
# Trong guest:
gdbserver 0.0.0.0:1234 /usr/bin/buggy_app
# Trong terminal HOST khác:
bash embedded_lab3/scripts/debug.sh
```

Rootfs được đóng gói bằng gen_init_cpio của kernel LAB01 để tạo device nodes mà không cần root. Toàn bộ lệnh đo và phiên bản công cụ nằm trong scripts/ và logs/environment.txt. Gói nộp chứa initramfs nhưng không chứa zImage/DTB trùng lặp; dùng các file LAB01 trong repo.

2 GDB remote và ba lỗi ứng dụng

Chương trình gốc được giữ riêng, không dựa vào việc nó phải crash. Phiên GDB đã kết thúc bình thường dù có undefined behavior. Comment “Null pointer dereference” trong mẫu không khớp phép truy cập vượt mảng thực tế.

Lỗi	Bằng chứng GDB	Nguyên nhân và cách sửa
Vượt mảng	buggy_app.c:9; i=5, len=5; data chỉ có 5 phần tử. Software watchpoint quan sát i: 5 → 6.	Vòng lặp dùng <=. Đổi thành i < len, kiểm tra pointer/length và quy ước số phần tử.
Use-after-free	buggy_app.c:18-19; buf vẫn ở 0x67da8, nhưng sau free chuỗi chỉ còn “g”.	Ownership đã kết thúc khi free. Trả buffer còn sống, kiểm tra malloc, rồi caller free sau khi dùng. Không dùng lại pointer đã free.
Signed integer overflow	buggy_app.c:24-25; a=b=100000; phép nhân 64-bit = 10000000000; int result = 1410065408.	Kết quả vượt INT_MAX. Dùng int64_t và ép kiểu trước phép nhân, hoặc kiểm tra overflow theo miền đầu vào. Signed overflow là undefined behavior.

Tại điểm vượt mảng: pc=0x10468, sp=0x7efffcf8, lr=0x105ec trong phiên đã ghi. Địa chỉ phụ thuộc build và lần chạy. GDB hiển thị backtrace process_data → main, create_message → main và calculate → main. Log đầy đủ nằm ở Phụ lục A và logs/gdb_session.txt.

Software watchpoint được chọn vì target QEMU/kernel không cung cấp hardware watchpoint phù hợp cho thí nghiệm này. Nó kiểm tra giá trị qua stepping nên chậm hơn; chỉ dùng cho chương trình nhỏ có kiểm soát.

3 Memory leak và bản sửa

Allocation	Block	Byte mất	Vị trí allocation
scores	100	2000	leaky_app.c:20
name	100	2100	leaky_app.c:18
message buffer	50	12800	leaky_app.c:36
Tổng definitely lost	250	16900	3 allocation contexts

delete_record chỉ free struct, làm mất các pointer tới name và scores. Vì struct đã được free chứ không còn là một block bị leak, Memcheck báo các allocation con là definitely lost trong lần chạy này; indirectly lost bằng 0. Vòng process_messages mất 50 buffer vì không free sau khi dùng.

Đọc allocation stack từ malloc xuống các caller: create_record → main hoặc process_messages → main. Stack ghi nơi cấp phát; nguyên nhân leak cần đối chiếu thêm với đường giải phóng trong source, không suy ra chỉ từ dòng malloc.

```
LEAK SUMMARY:
==14==    definitely lost: 16,900 bytes in 250 blocks
==14==    indirectly lost: 0 bytes in 0 blocks
==14==      possibly lost: 0 bytes in 0 blocks
==14==    still reachable: 0 bytes in 0 blocks
==14==         suppressed: 0 bytes in 0 blocks
==14==
==14== For lists of detected and suppressed errors, rerun with: -s
==14== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
```

Bản sửa giải phóng name và scores trước struct, free message buffer sau khi dùng, kiểm tra cấp phát và dọn các allocation đã thành công nếu bước sau thất bại. Source gốc và bản sửa là hai file riêng.

```
==28== Memcheck, a memory error detector
==28== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==28== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==28== Command: embedded_lab3/apps/leaky_app_fixed
==28== Parent PID: 2
==28==
==28==
==28== HEAP SUMMARY:
==28==     in use at exit: 0 bytes in 0 blocks
==28==   total heap usage: 352 allocs, 352 frees, 24,196 bytes allocated
==28==
==28== All heap blocks were freed -- no leaks are possible
==28==
==28== For lists of detected and suppressed errors, rerun with: -s
==28== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Quan sát bộ nhớ trên ARM

memory_observe_arm chạy cùng workload với hai chế độ leaky/fixed, chờ trước và sau workload để shell đọc /proc khi process còn sống. Đây là helper quan sát; không thay source gốc dùng làm baseline Valgrind.

Chế độ	Thời điểm	VmSize kB	VmRSS kB	VmData kB
leaky	trước	624	4	152
leaky	sau	624	4	152
fixed	trước	624	4	152
fixed	sau	624	4	152

Các counter không đổi trong workload nhỏ này. Allocator có thể dùng arena đã có và giữ lại vùng đã free; RSS cũng không đếm “byte leak” trực tiếp. Vì vậy bảng này không thay thế bằng chứng Memcheck. Memory map cho thấy các vùng text, data, heap và stack; output gốc nằm trong logs/arm_memory.txt.

4 Profiling bằng strace và perf

Giữ perf_target_original.c theo đề và perf_target.c cho bản đo. Bản đo kiểm tra allocation/I/O, dùng mkstemp để tránh dùng chung file /tmp, và thay đổi target tìm kiếm theo vòng lặp. Assembly trước/sau được lưu vì compiler đã hoist các truy vấn giống nhau thành một lần gọi, ngay cả khi chương trình dùng checksum.

Strace/timing HOST và ARM đều dùng 50 vòng heavy_compute. Perf dùng 2000 vòng và 997 Hz để tăng số mẫu, vì lượt 99 Hz trước đó chỉ ghi nhận ba symbol. Không so trực tiếp thời gian hai cấu hình workload khác nhau.

Syscall HOST	% syscall time	Số lần	Lỗi
munmap	45.15	3	0
unlink	25.42	1	0
write	19.06	70	0
mmap	4.35	14	0
brk	4.01	3	0

Đây là tỷ lệ trong thời gian syscall được strace thống kê, không phải tỷ lệ thời gian toàn chương trình. Dữ liệu đầy đủ ở strace_full.txt và strace_summary.txt. fprintf có buffering nên 10000 dòng không tạo 10000 lần write; HOST ghi khoảng 70 lần và ARM 76 lần trong các lượt đã lưu.

Symbol	Shared object	% mẫu theo trọng số
__ieee754_log_fma	libm.so.6	50.91
heavy_compute	perf_target	37.36
log@@GLIBC_2.29	libm.so.6	7.48
linear_search	perf_target	4.02
log@plt	perf_target	0.11

Bảng giữ đúng symbol mà perf xuất ra; log@plt là trampoline liên kết động, không phải một thuật toán độc lập. Hàm log của libm và heavy_compute chiếm phần lớn mẫu. Call graph nối libm về heavy_compute; chương trình thiên về CPU, không thể kết luận nghẽn I/O chỉ từ strace. Samples và event count đầy đủ nằm trong perf_report.txt; không có lost sample trong bộ cuối.

```
 Performance counter stats for '/home/kaymai/Documents/Codex/2026-10-28/tao/outputs/embedded-linux-labs/embedded_lab3/apps/perf_target 2000':

          1968.72 msec task-clock
               77      context-switches
                2      cpu-migrations
              372      page-faults
       6675660252      cycles
      15130981878      instructions
         17507083      cache-misses
       2205131891      branches
         19058893      branch-misses

      1.972418014 seconds time elapsed

      1.966806000 seconds user
      0.002998000 seconds sys
```

Counter instructions/cycles dùng để tính IPC cho workload này; cache-misses không phải một tỷ lệ nếu chưa có denominator phù hợp. IPC không có ngưỡng “tốt/xấu” chung cho mọi CPU và chương trình. Flame graph được dựng từ perf_stacks.txt bằng công cụ FlameGraph chính thức; repo giữ SVG, folded stacks và commit của công cụ.

HOST chạy native x86-64, còn ARM chạy dưới QEMU TCG; thời gian ARM chậm hơn không phải phép so sánh tốc độ hai CPU thật. Các syscall, ABI và cách link static/dynamic cũng khác. Bản ARM sau cùng trả checksum 99991021; nó được kiểm tra lại sau thay đổi workload.

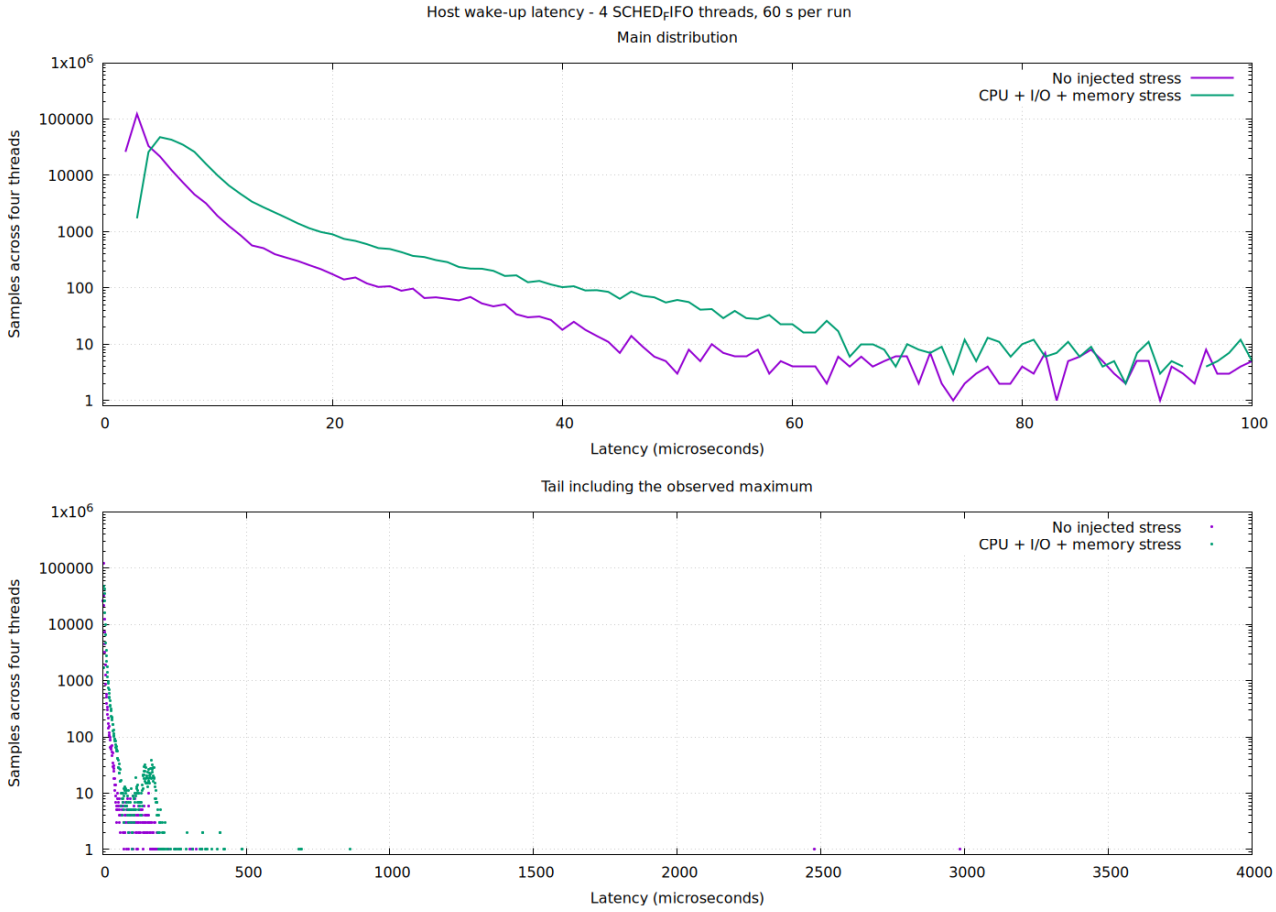
5 Latency trên kernel HOST

Đo trực tiếp trên HOST, không đo trong QEMU. Mỗi lượt 60 giây: 4 thread SCHED_FIFO priority 80, pin CPU 0-3, interval 1000 µs, mlockall, histogram 10000 µs. Histogram yêu cầu cùng interval nên distance đặt 0. Cyclictest tạm yêu cầu /dev/cpu_dma_latency=0 µs trong thời gian chạy; không thay sysctl hoặc cấu hình kernel.

Lượt baseline không chủ động tạo stress. Lượt có tải chạy stress-ng với 8 CPU worker, 2 I/O worker và 2 VM worker, vm-bytes=256M, timeout=70s; warm-up 5 giây trước đo. QEMU và HTTP server thử nghiệm được dừng trước bộ latency cuối. Desktop và các dịch vụ HOST vẫn hoạt động, không phải máy đo đã cô lập hoàn toàn.

Chỉ số	Không tạo stress	Có stress
Min (µs)	2	3
Avg gần đúng, gộp thread (µs)	4.36	8.88
Max quan sát (µs)	2982	863
P99 theo histogram (µs)	19	45
Số mẫu	239996	240000
Histogram overflow	0	0

Avg gộp được tính theo số cycle của từng thread từ Avg đã làm tròn trong JSON. P99 tính từ histogram gộp; Max lấy trực tiếp từ summary, không lấy bucket cuối. Kiểm tra tổng histogram cùng overflow bằng tổng số cycle.



Trục Y dạng log để nhìn được phần đuôi ít mẫu. Biểu đồ không cắt ở 200 µs vì làm vậy có thể giấu các spike lớn. Đây là phân phối của bốn thread gộp lại.

Kết quả một lượt có tải không bắt buộc có Max lớn hơn lượt baseline: Max chịu ảnh hưởng sự kiện hiếm và thời gian quan sát hữu hạn. So sánh đồng thời Avg, P99, Max và điều kiện đo. Không dùng một lần đo để tuyên bố hard real-time hoặc xác lập worst-case bound.

PREEMPT_RT chỉ được phân tích lý thuyết theo D.5; chưa có phép đo trên kernel RT. Kernel lowlatency không đồng nghĩa CONFIG_PREEMPT_RT=y. Với robot yêu cầu response dưới 500 µs, ưu tiên PREEMPT_RT kèm cấu hình priority/affinity, IRQ, bộ nhớ và kiểm thử trên phần cứng đích. Cyclictest chỉ đo wake-up latency, không bao gồm toàn bộ sensor-to-actuator response.

6 Trả lời câu hỏi phân tích

Câu 1 Chuỗi xử lý lệnh next

GDB HOST dùng debug symbols, line table và kiến trúc ARM để xác định nơi cần dừng tiếp theo. next là thao tác source-level step-over: GDB có thể phối hợp single-step và breakpoint tạm để đi qua lời gọi hàm. Không có quy tắc mọi next đều chỉ gửi đúng một packet “next”.

Các yêu cầu điều khiển được gửi qua GDB Remote Serial Protocol trên TCP, qua hostfwd vào gdbserver trong guest. Gdbserver dùng ptrace điều khiển process ARM; khi target dừng vì trap/signal, nó gửi stop reply. GDB lấy registers/memory và ánh xạ pc về source để hiển thị. Gdb-multiarch hỗ trợ ARM registers, ABI và instruction set; một GDB chỉ build cho x86 không mặc định làm được việc này. Phiên thực tế đã đọc pc/sp/lr, backtrace và software watchpoint.

Câu 2 Definitely lost và indirectly lost

Definitely lost nghĩa không còn đường pointer được detector nhận diện tới allocation. Indirectly lost là khi pointer tới allocation chỉ còn qua các block cũng đã bị mất. Trong bài này free(r) được gọi nhưng không free name/scores, nên Memcheck ghi 2100 và 2000 byte definitely lost, không phải indirectly lost. Nếu bản thân r bị leak và vẫn chứa pointer tới các allocation con, phân loại có thể chuyển sang gián tiếp.

Ngoài leak, Memcheck phát hiện invalid read/write, use-after-free, double/invalid free và sử dụng giá trị chưa khởi tạo. Leak check không thay thế kiểm tra lỗi số học; signed integer overflow cần phân tích miền giá trị hoặc công cụ bổ sung. Bản sửa thực tế có 352 allocs/352 frees và ERROR SUMMARY bằng 0.

Câu 3 Tối ưu heavy_compute

Phương án 1: dữ liệu darr không đổi qua các iteration, nên có thể tính tổng sqrt(x)*log(x+1) một lần rồi tái sử dụng hoặc nhân với số iteration nếu chấp nhận khác biệt rounding. Với 50 lượt, phần tính toán lặp có trần cải thiện lý thuyết gần 50 lần; tổng chương trình còn bị giới hạn bởi search, I/O và khởi tạo. Đây là ước lượng theo số phép tính, chưa phải speedup đã benchmark.

Phương án 2: dùng nhiều accumulator/loop unrolling để giảm dependency chain của phép cộng và hỗ trợ scheduling các phép toán độc lập; dữ liệu được duyệt liên tục để giữ locality. Mức tăng cần benchmark, không thể hứa 2x hay 4x từ số lane. Với Cortex-A9, NEON AArch32 không xử lý FP64 vector như mảng double hiện tại; muốn dùng SIMD float32 phải đánh giá sai số, dùng thư viện vector math phù hợp và kiểm tra lại kết quả. log vẫn là phép toán đắt, không có một instruction NEON “log” thay trực tiếp.

Trong bộ perf cuối, log/libm và heavy_compute là các hotspot thật. Việc thay đổi search query trong bản đo nhằm giữ workload có thật, không được trình bày như tối ưu giúp chương trình chạy nhanh hơn.

Câu 4 Latency spike và PREEMPT_RT

Nguồn 1 là hard IRQ và vùng tắt interrupt/preemption: tác vụ ưu tiên cao vẫn phải chờ. PREEMPT_RT chuyển phần lớn IRQ sang thread để scheduler điều khiển, nhưng các ngoại lệ và low-level raw critical sections vẫn còn.

Nguồn 2 là contention/priority inversion quanh lock. PREEMPT_RT chuyển nhiều spinlock sang sleeping lock dựa trên rtmutex và priority inheritance để tác vụ giữ lock được ưu tiên phù hợp; không biến mọi lock thành preemptible hoặc bỏ mọi đoạn bảo vệ.

Nguồn 3 là page fault, reclaim và hoạt động quản lý bộ nhớ. mlockall, prefault và tránh allocation/I/O không giới hạn trong vòng điều khiển giúp giảm biến thiên; PREEMPT_RT không tự loại bỏ mọi page fault. Ngoài ra SMI/firmware, power management và thiết bị có thể gây spike ngoài khả năng khắc phục hoàn toàn của scheduler. Số liệu Max/P99 của bài phải được đọc cùng cấu hình và tải, không suy rằng Avg thấp là bảo đảm deadline.

Câu 5 Chọn công cụ theo triệu chứng

Triệu chứng	Công cụ ưu tiên	Lý do và bước tiếp
Crash ngẫu nhiên sau vài giờ	GDB và core dump	Bắt signal, stack, registers; dùng Valgrind nếu nghi UAF/invalid access.
Chạy chậm dần theo thời gian	Valgrind + /proc	Kiểm tra leak và tăng bộ nhớ; bổ sung strace để tìm FD/syscall bị tích lũy hoặc blocking.
Đôi khi lỡ deadline	cyclictest	Đo phân phối/tail wake-up latency dưới tải; tiếp tục trace scheduler/IRQ nếu cần tìm nguyên nhân.
Một hàm làm hệ thống chậm	perf	Lấy mẫu CPU/call graph; dùng strace nếu nguyên nhân thực tế là chờ syscall/I/O.

7 Khó khăn và cách xử lý

Rootfs thiếu cấu hình resolver: gdbserver :1234 không phân giải được tên. Dùng địa chỉ số 0.0.0.0:1234; thêm DHCP script để guest có IP trước kết nối.

Crash khi profiling bằng sudo: mẫu gốc mở file /tmp cố định mà không kiểm tra fopen. File do user tạo trước đó cùng fs.protected_regular=2 khiến luồng mở dưới sudo có thể thất bại, rồi fprintf nhận NULL. Ban đầu chẩn đoán nhầm là lỗi perf và thử bản 6.8; cả hai đều thất bại. Sửa bằng mkstemp và xử lý lỗi I/O; bộ cuối dùng perf 7.0.14, các lệnh đều kết thúc thành công. Log trước sửa được giữ để thể hiện quá trình chẩn đoán.

Compiler thay đổi workload: kết quả search ban đầu không được dùng; sau khi thêm checksum, các truy vấn hệt nhau vẫn bị hoist. Đối chiếu objdump rồi thay đổi truy vấn mỗi vòng để bảo đảm thực thi công việc cần đo.

Giới hạn quyền: sandbox chặn ptrace và HOST đặt perf_event_paranoid=4. Strace/GDB chạy ngoài sandbox; perf và SCHED_FIFO chạy qua sudo. Không hạ perf_event_paranoid toàn hệ thống. File perf.data do root tạo được trả quyền sở hữu cho user sau phép đo.

Tài liệu tham khảo

GDB Remote Serial Protocol: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Remote-Protocol.html

Valgrind Memcheck manual: https://valgrind.org/docs/manual/mc-manual.html

Linux PREEMPT_RT differences: https://docs.kernel.org/core-api/real-time/differences.html

Arm C Language Extensions, AArch32 Neon: https://arm-software.github.io/acle/main/acle.html

FlameGraph tools: https://github.com/brendangregg/FlameGraph

Phụ lục A Toàn bộ phiên GDB

```
The target architecture is set to "arm".
0x00010340 in _start ()
Breakpoint 1 at 0x10580: file outputs/embedded-linux-labs/embedded_lab3/apps/buggy_app.c, line 29.

Breakpoint 1, main (argc=1, argv=0x7efffe94) at outputs/embedded-linux-
labs/embedded_lab3/apps/buggy_app.c:29
29  int main(int argc, char *argv[]) {
30      printf("=== Buggy App v1.0 — LAB-03 Debug Target ===\n");
33      int arr[5] = {10, 20, 30, 40, 50};
34      printf("[TEST1] Processing array of 5 elements\n");
$1 = {10, 20, 30, 40, 50}
pc             0x105cc             0x105cc <main+96>
sp             0x7efffd10          0x7efffd10
lr             0x478f0             293104
Breakpoint 2 at 0x10468: file outputs/embedded-linux-labs/embedded_lab3/apps/buggy_app.c, line 9.

Breakpoint 2, process_data (data=0x7efffd20, len=5) at outputs/embedded-linux-
labs/embedded_lab3/apps/buggy_app.c:9
9           sum += data[i];                 /* Out-of-bounds khi i==len */

=== ARRAY OUT OF BOUNDS ===
$2 = 5
$3 = 5
$4 = {10, 20, 30, 40, 50}
$5 = (int *) 0x7efffd34
Watchpoint 3: i
8       for (int i = 0; i <= len; i++) {   /* Bug: <= thay vi < */

Watchpoint 3: i

Old value = 5
New value = 6
0x00010494 in process_data (data=0x7efffd20, len=5) at outputs/embedded-linux-
labs/embedded_lab3/apps/buggy_app.c:8
8       for (int i = 0; i <= len; i++) {   /* Bug: <= thay vi < */
#0  0x00010494 in process_data (data=0x7efffd20, len=5) at outputs/embedded-linux-
labs/embedded_lab3/apps/buggy_app.c:8
#1  0x000105ec in main (argc=1, argv=0x7efffe94) at outputs/embedded-linux-
labs/embedded_lab3/apps/buggy_app.c:35
pc             0x10494             0x10494 <process_data+84>
sp             0x7efffcf8          0x7efffcf8
lr             0x105ec             67052
Breakpoint 4 at 0x10504: file outputs/embedded-linux-labs/embedded_lab3/apps/buggy_app.c, line 18.

Breakpoint 4, create_message (text=0x47894 "Hello Embedded Linux") at outputs/embedded-linux-
labs/embedded_lab3/apps/buggy_app.c:18
18      free(buf);

=== BEFORE FREE ===
$6 = 0x67da8 "Hello Embedded Linux"
0x67da8:    "Hello Embedded Linux"
19      return buf;   /* Bug: tra ve pointer da bi free */

=== AFTER FREE ===
$7 = 0x67da8 "g"
0x67da8:    "g"
#0  create_message (text=0x47894 "Hello Embedded Linux") at outputs/embedded-linux-
labs/embedded_lab3/apps/buggy_app.c:19
#1  0x0001060c in main (argc=1, argv=0x7efffe94) at outputs/embedded-linux-
labs/embedded_lab3/apps/buggy_app.c:39
Breakpoint 5 at 0x10530: file outputs/embedded-linux-labs/embedded_lab3/apps/buggy_app.c, line 24.

Breakpoint 5, calculate (a=100000, b=100000) at outputs/embedded-linux-
labs/embedded_lab3/apps/buggy_app.c:24
24      int result = a * b;   /* Bug: a*b co the overflow int */

=== INTEGER OVERFLOW ===
$8 = 100000
$9 = 100000
$10 = 10000000000
25      printf("calculate(%d, %d) = %d\n", a, b, result);
```

```
$11 = 1410065408
r0             0x186a0             100000
r1             0x186a0             100000
r2             0x186a0             100000
r3             0x540be400          1410065408
r4             0x1                 1
r5             0x5fde0             392672
r6             0x1                 1
r7             0x7efffe94          2130706068
r8             0x7efffe9c          2130706076
r9             0x62010             401424
r10            0x1                 1
r11            0x7efffd0c          2130705676
r12            0x4                 4
sp             0x7efffcf8          0x7efffcf8
lr             0x10648             67144
pc             0x10540             0x10540 <calculate+36>
cpsr           0x20070010          537329680
fpscr          0x0                 0
#0  calculate (a=100000, b=100000) at outputs/embedded-linux-labs/embedded_lab3/apps/buggy_app.c:25
#1  0x00010648 in main (argc=1, argv=0x7efffe94) at outputs/embedded-linux-
labs/embedded_lab3/apps/buggy_app.c:44
[Inferior 1 (process 82) exited normally]
```

Phụ lục B Allocation stack trước sửa

```
==14== Memcheck, a memory error detector
==14== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==14== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==14== Command: embedded_lab3/apps/leaky_app
==14== Parent PID: 2
==14==
==14==
==14== HEAP SUMMARY:
==14==     in use at exit: 16,900 bytes in 250 blocks
==14==   total heap usage: 352 allocs, 102 frees, 24,196 bytes allocated
==14==
==14== 2,000 bytes in 100 blocks are definitely lost in loss record 1 of 3
==14==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==14==    by 0x109280: create_record (leaky_app.c:20)
==14==    by 0x10940E: main (leaky_app.c:51)
==14==
==14== 2,100 bytes in 100 blocks are definitely lost in loss record 2 of 3
==14==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==14==    by 0x10924D: create_record (leaky_app.c:18)
==14==    by 0x10940E: main (leaky_app.c:51)
==14==
==14== 12,800 bytes in 50 blocks are definitely lost in loss record 3 of 3
==14==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==14==    by 0x109311: process_messages (leaky_app.c:36)
==14==    by 0x109481: main (leaky_app.c:59)
==14==
==14== LEAK SUMMARY:
==14==    definitely lost: 16,900 bytes in 250 blocks
==14==    indirectly lost: 0 bytes in 0 blocks
==14==      possibly lost: 0 bytes in 0 blocks
==14==    still reachable: 0 bytes in 0 blocks
==14==         suppressed: 0 bytes in 0 blocks
==14==
==14== For lists of detected and suppressed errors, rerun with: -s
==14== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
```

Phụ lục C Kết quả ARM và latency theo thread

```
/ # time /tmp/perf_target_arm 50
=== Perf Target App — LAB-03 ===
[1] CPU-intensive computation...
    Result: 11433083166.074631
[2] Linear search (1000 times)...
    Search checksum: 99991021
[3] I/O intensive (write 10000 lines)...
Done.
real    0m 1.93s
user    0m 1.89s
sys 0m 0.03s
/ # strace -c /tmp/perf_target_arm 50
=== Perf Target App — LAB-03 ===
[1] CPU-intensive computation...
    Result: 11433083166.074631
[2] Linear search (1000 times)...
    Search checksum: 99991021
[3] I/O intensive (write 10000 lines)...
Done.
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 43.28    0.007636        3818         2           munmap
 20.45    0.003608          47        76           write
 12.50    0.002205        2205         1           execve
  4.40    0.000777         777         1           rseq
  4.11    0.000725         145         5           brk
  3.25    0.000574         574         1           readlinkat
  1.64    0.000289         144         2           mmap2
  1.56    0.000276         276         1           ioctl
  1.51    0.000267         267         1           set_tls
  1.45    0.000255         127         2           statx
  1.33    0.000234         117         2           clock_gettime64
  0.95    0.000168         168         1           ugetrlimit
  0.94    0.000166         166         1           set_tid_address
  0.94    0.000165         165         1           mprotect
  0.83    0.000146         146         1         1 getrandom
  0.74    0.000131         131         1           set_robust_list
  0.10    0.000017          17         1           unlink
  0.02    0.000004           4         1           close
  0.00    0.000000           0         1           fcntl64
  0.00    0.000000           0         1           openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.017643         171       103         1 total
/ # echo __VERIFIED_PROFILE_END__
```

Thống kê chi tiết theo từng CPU/thread nằm trong cyclic_idle.json, cyclic_loaded.json và latency_summary.json. Các file log còn lại và dữ liệu perf/histogram được nộp kèm để kiểm tra lại, không thay thế bằng output mẫu trong đề. MSSV và nhóm/lớp cần điền trước khi nộp chính thức.